



PRÉ-RENTRÉE 2026-2027

Séance 2 - Fiche enseignant

Boucles, compteurs et accumulateurs

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Corrigé, différenciation et pilotage

Objectifs

- comprendre les valeurs produites par range ;
- choisir entre for et while ;
- utiliser compteur et accumulateur ;
- éviter les erreurs de borne ;
- justifier une terminaison simple ;

Déroulé minute par minute

- 0-10 rituel ;
- 10-30 range et bornes ;
- 30-50 compteurs/accumulateurs ;
- 50-65 for/while ;
- 65-70 pause ;
- 70-100 atelier ;
- 100-112 différenciation ;
- 112-120 invariant et exit ;

Rituel prêt à l'emploi

Question 1

Écrire, dans l'ordre, les valeurs produites par :

```
range(2, 9, 3)
```

Réponse attendue : 2, 5, 8.

Question 2

Tracer puis donner la valeur finale de `s` :

```
s = 0
for i in range(4):
    s = s + i
```

Réponse attendue : `s = 6`, car on ajoute $0 + 1 + 2 + 3$.

Notions de cours

Boucle bornée

```
for i in range(3):
    print(i)
```

produit `0`, `1`, `2`. La borne supérieure est exclue.

Boucle non bornée

```
while condition:
    # corps
```

Elle convient lorsque le nombre de répétitions n'est pas connu à l'avance. Il faut identifier un **variant** qui évolue vers l'arrêt.

Schémas de base

- compteur : `compteur += 1` ;
- accumulateur : `somme += valeur` ;
- recherche d'extremum : initialiser avec le premier élément, puis comparer ;
- parcours : traiter chaque élément exactement une fois.

Activité commune - prévoir avant d'exécuter

```
s = 0
for i in range(1, 4):
    s = s + i
```

Tour	☒ 1	☒ avant	☐ après
1			
2			

Tour	☐	☒ avant	☐ après
3			

Choisir la boucle

Situation	for ou while ?	Justification
parcourir une liste		
demandeur un mot de passe jusqu'à réussite		
afficher 10 nombres		
lire des valeurs jusqu'au mot fin		

Parcours Fondations - Ahmad

1. Écrire les valeurs de `i` pour `range(5)`, `range(2, 6)` et `range(10, 3, -2)`.
2. Compléter un programme qui calcule la somme de 1 à `n`.
3. Compter le nombre de valeurs positives d'une liste.
4. Calculer le maximum sans utiliser `max`.
5. Corriger une boucle `while` qui ne se termine jamais.

Parcours Fiabilisation - Ahmed

1. Écrire une fonction de test manuel pour vérifier une somme cumulée.
2. Calculer simultanément somme, effectif, minimum et maximum en un seul parcours.
3. Traiter proprement la liste vide.
4. Expliquer un invariant du calcul de somme.
5. Écrire une boucle `while` dont le variant est explicite.

Corrigé essentiel

- `range(5)` : 0,1,2,3,4 ; `range(2, 6)` : 2,3,4,5 ; `range(10, 3, -2)` : 10,8,6,4 ;
- somme de 1 à `n` : initialiser `somme=0`, ajouter chaque entier ;
- maximum : initialiser avec le premier élément, puis remplacer si une valeur plus grande est rencontrée ;
- une boucle `while` doit modifier une donnée intervenant dans sa condition.

Consignes prêtes à dire

- « Avant d'exécuter, écris ce que tu prévois. »
- « Une réponse sans contrôle reste une hypothèse. »
- « Le bug utile est celui que l'on peut reproduire. »
- « Ne change qu'une chose à la fois, puis relance les tests. »
- « Explique le rôle de cette variable sans lire le code mot à mot. »

Points de vigilance

- ne pas transformer l'activité en copie de code projeté ;
- vérifier que les deux élèves alternent pilote et navigateur ;
- ne pas confondre réussite après aide D/E et autonomie ;
- demander un test sur les bornes ;
- conserver le fichier final et le journal des erreurs.

Indicateurs de réussite

Élève	Prévision exacte	Code exécutable	Tests pertinents	Explication	Aide maximale
Ahmad BELDI					
Ahmed BENHADJ SALEM					